Tips matematica

Francisco Carthery

28 de enero de 2026

1. Counting Divisors

Si tenemos varias queries donde debemos calcular la cantidad de divisores de un numero podemos:

- Usar la funcion τ factorizando cada numero. Complejidad $\boldsymbol{O(nlog(n))}$ precalculando la criba o $\boldsymbol{O(n\sqrt{n})}$
- Creamos un vector del tamaño del maximo numero que podemos recibir (**n**) y similar a la Criba, para cada numero **i** desde 1 hasta n hacemos un bucle que vaya de **i** en **i** y sumamos 1 en ese valor del vector que creamos representando que ese numero es divisible por i. Al final en cada posicion del vector vamos a tener cuantos divisores tiene este numero. La complejidad de la estrategia es armonica: $\boldsymbol{O(nlog(n))}$

2. Common Divisors

Nos dan un vector de numeros y tenemos que elegir dos numeros tales que su **gcd sea maximo**. Hay dos opciones:

- Creamos un vector tipo frecuencia de divisores y a cada numero lo factorizamos en $\boldsymbol{O(\sqrt{n})}$ y sumamos sus divisores al vector frecuencia. El mayor divisor tal que su frecuencia sea >1 va a ser la respuesta. La complejidad es $\boldsymbol{O(n\sqrt{n})}$ por lo que entra justo ($2 \cdot 10^8$ operaciones).
- Otra opcion mas estetica es primero crear una frecuencia de los numeros que nos pasan en la entrada y despues, con una idea similar a la de la Criba, por cada posible divisor **k**, vamos de **k** en **k** y sumamos la frecuencia de esos multiplos de **k**. Como los unicos numeros que van a tener a **k** como divisor son sus multiplos, si en la suma total obtenemos un valor >1 ya sabemos que **k** es una posible respuesta. Si al bucle de los divisores lo hacemos de mayor a menor, la primera vez que encontremos un valor que tenga mas de un multiplo ya sabemos que va a ser el mayor y es la respuesta al problema. La complejidad es $\boldsymbol{O(nlog(n))}$

```cpp
const int MAXN = 1e6 + 1;
int n, x; cin >> n;
vector<int> v(MAXN);

forn(i, n){
    cin >> x;
    v[x]++;
}

dfor(i, MAXN){
    int val = 0;
    for(int j = i; j < MAXN; j += i){
        val += v[j];
    }

    if(val > 1){
        cout << i << '\n';
        return 0;
    }
}
```

Listing 1: Solucion de la segunda forma

3. Divisor Analysis

Nos dan un numero en forma de su factorizacion en primos donde cada factor primo puede ser masivo (x^k donde $2 \leq x \leq 10^6$ y $1 \leq k \leq 10^9$) y nos piden calcular la cantidad de divisores, su suma y su producto.

Para el producto usamos la funcion τ y para la suma usamos la funcion σ. El problema esta en el producto. Existe la siguiente formula para el producto de los factores:

$$\prod d = \prod_{i=1}^{k} p_i^{\frac{\alpha_i(\alpha_i+1)}{2} \frac{\tau(n)}{\alpha_i+1}} = N^{\frac{\tau(n)}{2}}$$

El temario es que para calcular una potencia $mod\ m$ tenemos que usar el Fermat Little Theorem:

$$x^n\ mod\ m = x^{n\ mod\ (m-1)} mod\ m$$

Es decir que tendriamos que calcular la funcion τ ($mod\ m$ - 1) pero m - 1 no es coprimo con 2 entonces no existe el inverso modular de 2 mod (m - 1). Para resolver este problema hay dos formas:

- Usar las propiedades de la funcion τ y ya calcular directamente $\frac{\tau(n)}{2}$ $mod\ m$ - 1.
- Un truco que podemos usar es calcular $\tau\ mod\ 2 \cdot (m$ - $1)$ entonces cuando τ es par (solo en un caso especial es impar) la podemos dividir por 2 tranquilamente y nos queda en mod (m - 1) como queriamos.

4. Counting Coprime Pairs

Nos dan un vector de hasta 10^5 elementos y tenemos que decir la cantidad de pares de numeros coprimos que existen. Debido a que los numeros de la lista pueden ser como maximo 10^6, la mayor cantidad de factores primos distintos que podria tener un numero es es $2 \cdot 3 \cdot 5 \cdot 7 \cdot 11 \cdot 13 \cdot 17 \leq 10^6$.

Sabiendo esto lo que podemos hacer es recorrer la lista y por cada numero lo factorizamos en primos y nos fijamos de los numeros que ya recorrimos (los de la izquierda), con cuantos no compartimos ningun primo y sumamos ese valor a la respuesta. Para ver con cuantos numeros no compartimos primos, a medida que recorremos la lista vamos generando con los valores ya recorridos una frecuencia de sus divisores, entonces luego, utilizando Inclusion-Exclusion podemos hacer el conteo.

Debido a que no nos interesa la cantidad de veces que aparezca cada primo, sino si esta o no en la factorizacion, como mencionamos antes los numeros de la lista pueden tener como maximo 7 factores primos distintos por lo que la complejidad de la solucion es $\boldsymbol{O(n \cdot 2^7)}$ ya que por cada numero hacemos una Inclusion-Exclusion.

5. Binomial Coefficients

Hay 10^5 queries donde nos pasan dos numeros ($0 \leq b \leq a \leq 10^6$) y nos piden calcular el coeficiente binomial:

$$\binom{a}{b} = \frac{a!}{b!(a-b)!}\ mod\ m$$

Para hacer esto se precalculan todos los factoriales y e inversos factoriales hasta el maximo numero que nos puedan pasar y luego utilizando la formula anterior calculamos el resultado.

El detalle interesante es que para precalcular de forma eficiente todos los inversos factoriales y no hacer **n** exponenciaciones binarias lo que se hace es solamente calcular el inverso del maximo factorial $maxn!^{-1} mod m$ y si sabemos el valor de $\frac{1}{(k+1)!}$, multiplicando por $(k+1)$ podemos obtener el valor de $\frac{1}{k!}$. Utilizando este truquito nos ahorramos un $\boldsymbol{O(log(n))}$ quedando la complejidad final en $\boldsymbol{O(n)}$

6. Distributing Apples

Este problema es el famoso **Stars and Bars** donde nos piden la cantidad de maneras en las que podemos ubicar n pelotas en k cajas. La formula es:

$$\binom{n+k-1}{n}$$

La intuicion para llegar a esta formula es pensar en que tenemos un string con n estrellas y $k-1$ barras. Por ejemplo, para $k=4$ y $n=5$ una posibilidad es ★|★★||★★. La cantidad de estrellas entre cada barra representa la cantidad de pelotas en esa caja. Podemos ver que la respuesta al problema es la cantidad de permutaciones con repeticion de este string, es decir, $\frac{(n+k-1)!}{n!(k-1)!}$, ya que como es indistinto el orden relativo de las pelotas y las barras, tenemos que descontar esto del total. Esta formula es exactamente el coeficiente binomial de arriba.

7. Christmas Party

Este problema se conoce como **Counting Derangements**, donde tenemos que decir cuantas permutaciones de tamaño **n** existen que no tengan *puntos fijos*. Se puede resolver usando Inclusion-Exclusion pero una solucion mas corta es usando dp.

En cada posicion P_i puede haber *n - 1* posibles valores, es decir, todos menos i. Digamos que j es el elemento que vamos a poner en P_i. Aca se nos presentan dos casos:

- Colocamos i en una posicion distinta que P_j. En este caso nos quedan *n - 1* elementos y *n - 1* posiciones donde existe **exaxtamente una** posicion donde no puede ir cada elemento. Esto ya que para todas las posiciones $P_k \neq P_j$ el elemento es k mientras que para P_j justamente el elemento prohibido es i porque pusimos eso como hipotesis. Este caso se nos reduce al **mismo problema** con un elemento menos.

- Colocamos i en P_j. En este caso como tenemos la posicion final tanto de i como de j, nos quedan *n - 2* elementos y *n - 2* posiciones por lo que tenemos el **mismo problema** con dos elementos menos.

Debido a que por cada elemento tenemos las dos opciones, el resultado final es la suma de ambas.

$$f(n) = (n-1)(f(n-1) + f(n-2))$$

8. Permutation Rounds

Nos dan una lista con los numeros de 1 a n y una permutacion de tamaño n y nos dicen que en cada round movemos cada elemento de acuerdo a la permutacion. Lo que tenemos que decir es cuantos rounds van a pasar hasta que la lista original vuelva a estar ordenada. El problema es sacar el tamaño cada uno de los ciclos de la permutacion y hacer el *minimo comun multiplo* entre los tamaños.

La unica curisidad del problema es que como podemos llegar a tener muchos tamaños distintos, tenemos que expresar el mcm *mod* $10^9 + 7$. Al tener esta restriccion no podemos usar *lcm()* de C++. En su lugar tenemos que factorizar cada uno de los numeros y quedarnos de cada primo que aparezca su mayor exponente y al final calculamos la respuesta haciendo el producto de esos primos.

9. Bracket Sequences 2

Nos piden la cantidad de secuencias de paréntesis validas de tamaño n que empiecen con un prefijo que se nos da de antemano. Incialmente tenemos que ver si la secuencia que nos dan no tiene un prefijo invalido y cual es la diferencia entre la cantidad de paréntesis de apertura y cierre, a la que llamaremos **s**. Para resolverlo tenemos que modificar la fórmula de los números Catalanes:

$$C_n = \binom{2n}{n} - \binom{2n}{n+1}$$

Donde el primer coeficiente es la cantidad de secuencias posibles sin restricción y el segundo es la cantidad de secuencias invalidas. En el caso del problema, siendo **k** la longitud de la secuencia que buscamos construir, debemos encontrar la cantidad de secuencias que tengan $\frac{k-s}{2}$ paréntesis de apertura y $\frac{k+s}{2}$ de ciere. Si hacemos el mismo razonamiento que con los números Catalanes (leer Laaksonen), de tomar el primer prefijo invalido e invertirlo, notamos que el primer prefijo invalido tiene: $apertura - cierre = -s - 1$ ya que tenemos que cerrar s parentesis que nos quedaron del prefijo. Invirtiendo el prefijo $apertura - cierre = s + 1$, Luego, la formula final queda:

$$C_k = \binom{k}{\frac{k-s}{2}} - \binom{k}{\frac{k-s}{2} + s + 1}$$

10. Counting Necklaces

Nos piden la cantidad de formas distintas de construir un collar con n cuentas de m colores. Esto se resuelve con el **Burnside's Lemma** cuya formula es:

$$\frac{1}{n}\sum_{k=1}^{n} c(k)$$

Donde $c(k)$ es la cantidad de collares que quedan fijos al rotar el collar k veces. La cantidad de collares que quedan fijos al rotar k veces es $m^{gcd(n,k)}$ ya que al rotar k veces van a quedar iguales los collares que esten compuestos de bloques iguales de tamaño $gcd(n, k)$. Entonces la formula final queda:

$$\frac{1}{n}\sum_{k=1}^{n} m^{gcd(n,k)}$$

Basicamente la logica de la formula es que, si cada collar tendria n rotaciones distintas, el resultado seria $\frac{m^n}{n}$ pero como hay collares que son periodicos y tienen menos rotaciones distintas, si lo dividimos por n estariamos contando menos de lo que hay. Para compensar esto, en lugar de elegir un denominador para cada collar en funcion de la cantidad de rotaciones distintas (seria complicado), hacemos que esos collares que tienen menos rotaciones distintas sumen mas de una vez (agrandamos el numerador lo cual es equivalente a reducir el denominador), compensando el resultado final.

11. Counting Grids

Nos piden la cantidad de formas de llenar una grilla de tamaño $n \times n$ donde cada cuadrado puede ser blanco o negro y dos grillas son consideradas distintas si no hay una rotacion que las vuelve iguales. Este problema es otro ejemplo del Burnside's Lemma.

Siempre tenemos 4 rotaciones distintas que podemos hacer a la matriz. Para el caso de n **par**, si giramos 90° o 270° tenemos $2^{\frac{n^2}{4}}$ combinaciones distintas ya que se podrian separar todas las celdas en grupos de 4 que si o si ser iguales porque estan en un mismo ciclo en las rotaciones. Si giramos 180° tenemos $2^{\frac{n^2}{2}}$ ya que ahora a las celdas las separamos en grupos de 2 que tienen que ser iguales. Por ultimo, para 0°, tenemos 2^{n^2} ya que todas las posibles grillas son distintas si no rotamos.

Aplicando esto a la formula:

$$\frac{1}{n}\sum_{k=1}^{n} c(k) = \frac{2^{n^2} + 2^{\frac{n^2}{4}} + 2^{\frac{n^2}{2}} + 2^{\frac{n^2}{4}}}{4}$$

Por ultimo, para el caso de n **impar**, la celda del medio no es afectada por las rotaciones por lo que la forma mas simple de pensarlo (para mi xd) es restar 1 a n^2 olvidandonos del elemento central, hacemos exactamente lo mismo que si n fuera par, y al final, multiplicar por 2 todas las combinaciones anteriores ya que el elemento central puede ser blanco o negro siempre:

$$\frac{1}{n}\sum_{k=1}^{n} c(k) = \frac{2^{n^2-1} + 2^{\frac{n^2-1}{4}} + 2^{\frac{n^2-1}{2}} + 2^{\frac{n^2-1}{4}}}{2}$$

12. Throwing Dice

Nos piden la cantidad de maneras de obtener un numero $n \leq 10^{18}$ tirando un dado. El problema se resuelve mediante *exponenciacion de matrices*.

La clave es que el problema se puede expresar como una **recurrencia lineal**

$$f(n) = f(n-1) + f(n-2) + f(n-3) + f(n-4) + f(n-5) + f(n-6)$$

Si tenemos una recurrencia lineal de la forma:

$$f(n) = c_1 f(n-1) + c_2 f(n-2) + \cdots + c_k f(n-k)$$

Siempre existe una matriz X tal que

$$X \cdot \begin{bmatrix} f(i) \\ f(i+1) \\ \vdots \\ f(i+k-1) \end{bmatrix} = \begin{bmatrix} f(i+1) \\ f(i+2) \\ \vdots \\ f(i+k) \end{bmatrix}$$

La forma general de esta matriz es:

$$X = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ c_k & c_{k-1} & c_{k-2} & \cdots & c_1 \end{bmatrix}$$

Es decir, al multiplicar el vector de los valores actuales por X, lo que hacemos es correr todos los valores una posicion hacia arriba y calcular el nuevo valor de la ultima posicion usando la recurrencia.

La magia de esto es que si queremos avanzar m posiciones, usando exponenciacion de matrices podemos calcular X^m en $O(k^3 \log m)$ y luego multiplicar X^m por el vector inicial para obtener el valor en la posicion deseada. Para nuestro problema especifico, la matriz queda:

$$X = \begin{bmatrix} 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

13. Graph Paths 2

Nos piden, dado un grafo, el largo minimo de un camino que vaya desde el nodo 1 al n con exactamente k aristas. El problema se resuelve con exponenciacion de matrices similar al anterior.

Lo interesante aca es que hay que modificar la operacion de multiplicacion de matrices. En lugar de hacer una suma de productos para calcular cantidad de formas de comunicar dos nodos (lo clasico), tenemos que hacer un minimo entre sumas. Es decir, si tenemos dos matrices A y B, la multiplicacion modificada seria:

$$C_{i,j} = \min_k (A_{i,k} + B_{k,j})$$

De esta manera, en todo momento llevamos el camino minimo entre dos nodos. Al hacer la exponenciacion de matrices con esta nueva operacion, en la posicion (1, n) de la matriz resultante vamos a tener el camino minimo entre el nodo 1 y el nodo n con exactamente k aristas.

14. Sum of Four Squares

Nos dan 1000 queries donde nos pasan un numero $n \leq 10^7$ y tenemos que dar cuatro enteros positivos a, b, c, d tales que $n = a^2 + b^2 + c^2 + d^2$. Por el teorema de Lagrange, sabemos que siempre existe una solucion para cualquier numero natural.

Otro dato importante es que por otro teorema, sabemos que un numero se puede expresar como suma de tres cuadrados si y solo si no es de la forma $4^a(8b + 7)$, es decir, es muy poco comun.

Una solucion es primero precalcular todas las posibles duplas (a, b) tales que $a^2 + b^2 \leq 10^7$. Con esto, iterando con i hasta $\sqrt{n}$ podemos saber si un numero x se puede expresar como suma de tres cuadrados buscando si $x - i^2$ existe en las duplas precalculadas.

La observacion clave esta en que como los numeros que si o si necesitan 4 cuadrados son poco comunes, podemos iterar ese 4to cuadrado d y por cada intento probar si el numero restante $n - d^2$ se puede expresar como suma de tres cuadrados en $O(\sqrt{n})$. Es altisimamente probable que encontremos una solucion en pocos intentos, por lo que la solucion es eficiente en la practica.

Especificamente, entre 0 y 10^7 hay 100 numeros que necesitan mas mas de 65 iteraciones para encontrar la solucion y si agarran los peores 1000 numeros la cantidad de iteraciones promedio entre queries es 29. Una curiosidad es que si en las primeras $2^k + 1$ iteraciones no encontramos la solucion, hay que ir directo a la iteración $2^{k+1} + 1$ ya que en base a la obervacion veo que en el medio nunca encuentra solucion (no se porque). Aunque da perfecto en tiempo la solucion original, en base a esta observacion se podria optimizar solo iterando potencias de 2, logrando un maximo de 10 iteraciones en el peor caso.